Ingeniería de Datos
Databricks
2026-2

Contexto

Una tienda necesita almacenar sus ventas, corregir errores y automatizar la preparación de sus reportes.

Cree un conjunto de al menos 15 registros con los campos id_venta, fecha, producto, categoria, cantidad y precio_unitario. Incluya intencionalmente una venta duplicada, una venta sin producto y una venta con cantidad negativa.

Trabaje en su cuenta de Databricks Free Edition, utilice cómputo serverless y el catálogo disponible en su workspace. Para cada ejercicio, responde primero las preguntas de investigación con tus propias palabras e incluye las fuentes consultadas.

1. Almacenar y consultar datos en el catálogo

Preguntas de investigación

- ¿Qué es Unity Catalog en Databricks?
- ¿Qué diferencia hay entre un DataFrame de Spark y una tabla Delta?

Ejercicio

La tienda necesita conservar sus ventas para consultarlas desde diferentes notebooks.

1. Crea un esquema llamado ingenieria_datos_tunombre dentro del catálogo disponible.
2. Crea un notebook llamado 01_cargar_ventas.
3. Construye un DataFrame con las ventas y guárdalo como una tabla Delta llamada ventas_bronze.
4. Ubica la tabla en el explorador del catálogo e identifica sus columnas y tipos de datos.
5. Desde otro notebook, lee la tabla utilizando su nombre completo: catalogo.esquema.tabla.
6. Consulta cuántos registros contiene y muestra las ventas de una categoría que elijas.

Entrega: respuestas de investigación, código de creación y lectura, captura de la tabla en el catálogo y resultados de las consultas.

2. Transformar datos y aplicar reglas de calidad

Ejercicio

La tienda detectó errores en sus ventas y necesita una versión confiable para elaborar reportes.

1. Crea un notebook llamado 02_transformar_ventas y lee ventas_bronze.
2. Elimina registros idénticos duplicados.
3. Descarta ventas sin producto, con cantidad menor o igual a cero o con precio unitario menor o igual a cero.
4. Agrega una columna total_venta, calculada como cantidad por precio unitario.
5. Guarda el resultado en una tabla Delta llamada ventas_silver.
6. Genera una tabla llamada ventas_por_categoria con el valor total vendido por categoría.
7. Ejecuta nuevamente el notebook y comprueba que las tablas de salida no acumulen registros duplicados por la repetición del proceso.

Entrega: código, cantidad de registros antes y después de la limpieza y resumen de ventas por categoría.

3. Automatizar el proceso mediante un job

Preguntas de investigación

- ¿Qué es un job en Databricks y para qué sirve?
- ¿Qué es una tarea y cómo se diferencia de un job?
- ¿Qué significa establecer dependencias entre tareas?
- ¿Qué información del historial de ejecuciones permite identificar un fallo?

Ejercicio

La tienda quiere ejecutar la carga y la transformación sin abrir manualmente cada notebook.

1. Crea un job llamado job_ventas_tunombre.
2. Configura una primera tarea que ejecute 01_cargar_ventas.

3. Agrega una segunda tarea que ejecute 02_transformar_ventas, únicamente después de que la primera termine correctamente.
4. Ejecuta el job manualmente y revisa el estado y la duración de cada tarea.
5. Introduce un error controlado en la primera tarea, por ejemplo, una referencia a una columna inexistente, y ejecuta nuevamente el job.
6. Describe qué ocurrió con la segunda tarea y cómo identificaste la causa del fallo.
7. Corrige el error y deja una ejecución final exitosa.

Entrega: respuestas de investigación, captura de las dependencias entre tareas, evidencia de la ejecución fallida y de la ejecución exitosa, y explicación del error.

4. Construir un pipeline declarativo

Preguntas de investigación

- ¿Qué es un pipeline de datos y cuáles son sus etapas habituales?
- ¿Qué significa que un pipeline sea declarativo?
- ¿En qué se diferencian un job que ejecuta notebooks y un pipeline declarativo que define conjuntos de datos?

Ejercicio

La tienda quiere definir sus transformaciones y permitir que Databricks administre las dependencias entre los resultados.

1. Crea un pipeline ETL con Lakeflow Spark Declarative Pipelines y cómputo serverless.
2. Utiliza ventas_bronze como fuente.
3. Define una vista materializada llamada pipeline_ventas_limpias que aplique las reglas de limpieza del ejercicio 2 y calcule total_venta.
4. Define otra vista materializada llamada pipeline_ventas_por_categoria, dependiente de la anterior, que resuma el valor vendido por categoría.
5. Publica los resultados en tu catálogo y esquema.
6. Ejecuta el pipeline manualmente y revisa el grafo de dependencias.

7. Agrega una venta válida a ventas_bronze y vuelve a ejecutar el pipeline. No ejecutes nuevamente el notebook de carga durante esta prueba.
8. Comprueba cómo cambió el resumen y explica si el resultado coincide con el valor de la venta agregada.

Entrega: respuestas de investigación, código del pipeline, captura del grafo y comparación del resumen antes y después de agregar la venta.